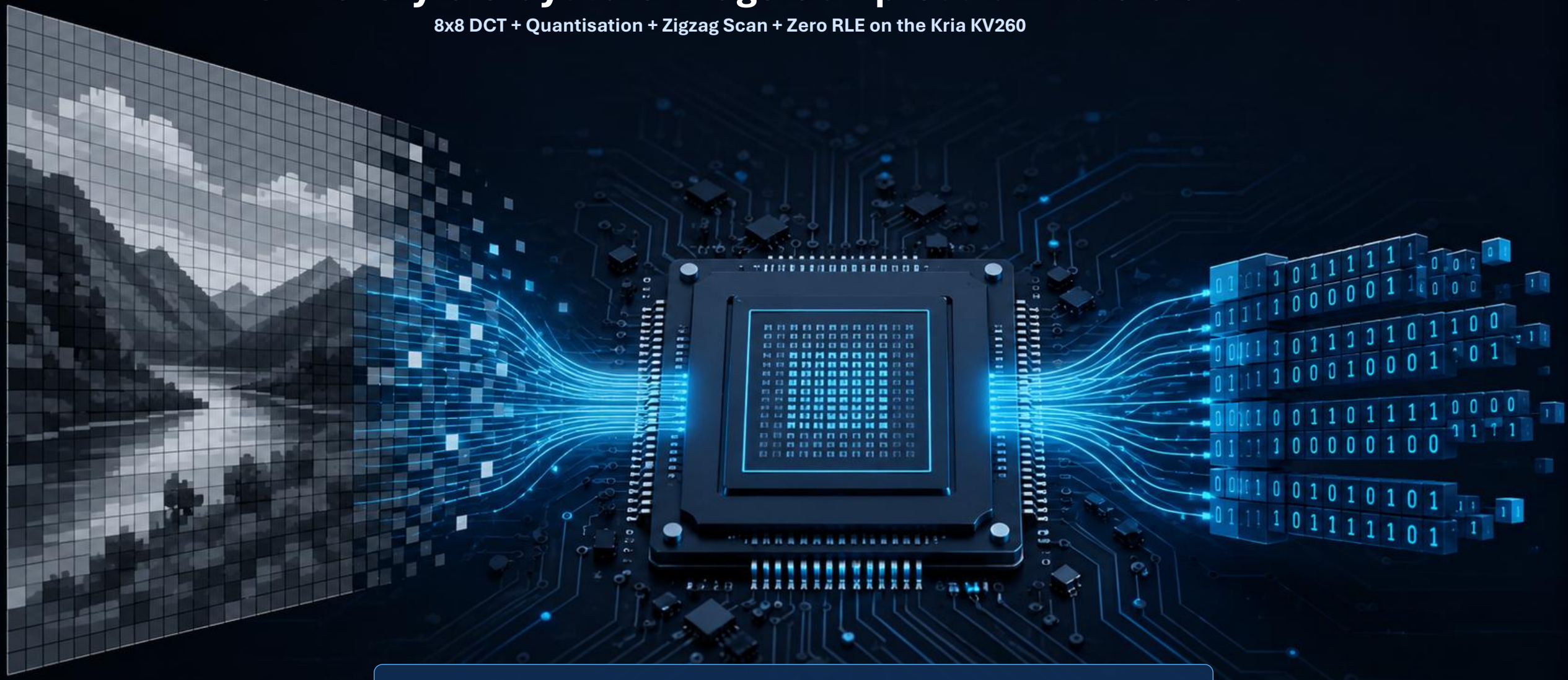
JPEG-Style Grayscale Image Compression Acceleration

8x8 DCT + Quantisation + Zigzag Scan + Zero RLE on the Kria KV260



Scope: Simplified JPEG-style block compression, not a full .jpg file encoder

Problem outline

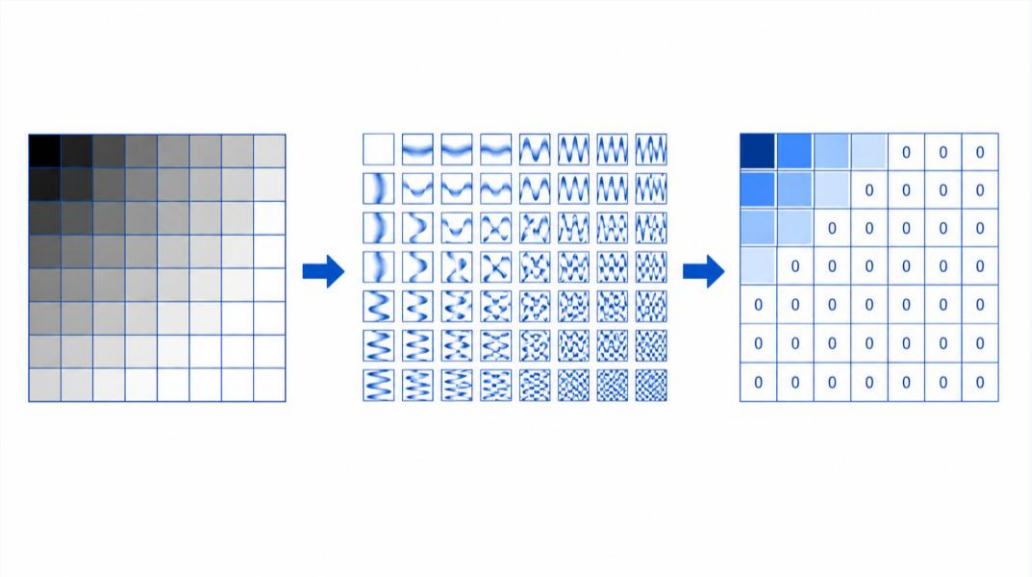
Why image compression is a suitable compute-acceleration task for COMP4601

The software bottleneck

JPEG-style compression repeatedly applies an 8x8 transform to every block of the image. The direct DCT contains many multiply-add operations, making it slow on a sequential CPU baseline.

The hardware opportunity

Each 8x8 block is independent, fixed-size, and processed with the same arithmetic pattern. This is well matched to FPGA pipelines, local buffers, loop unrolling, and array partitioning.



The project constraint

We avoid full JPEG file-format complexity. The project focuses on the core compression-side kernel: DCT, quantisation, zigzag ordering, and zero run-length encoding.

Expected output of the project

A working KV260 accelerator with CPU baseline, FPGA kernel timing, end-to-end timing, compression ratio, PSNR/MSE quality metrics, resource usage, and risk-aware discussion of transfer overhead.

Key design decision:

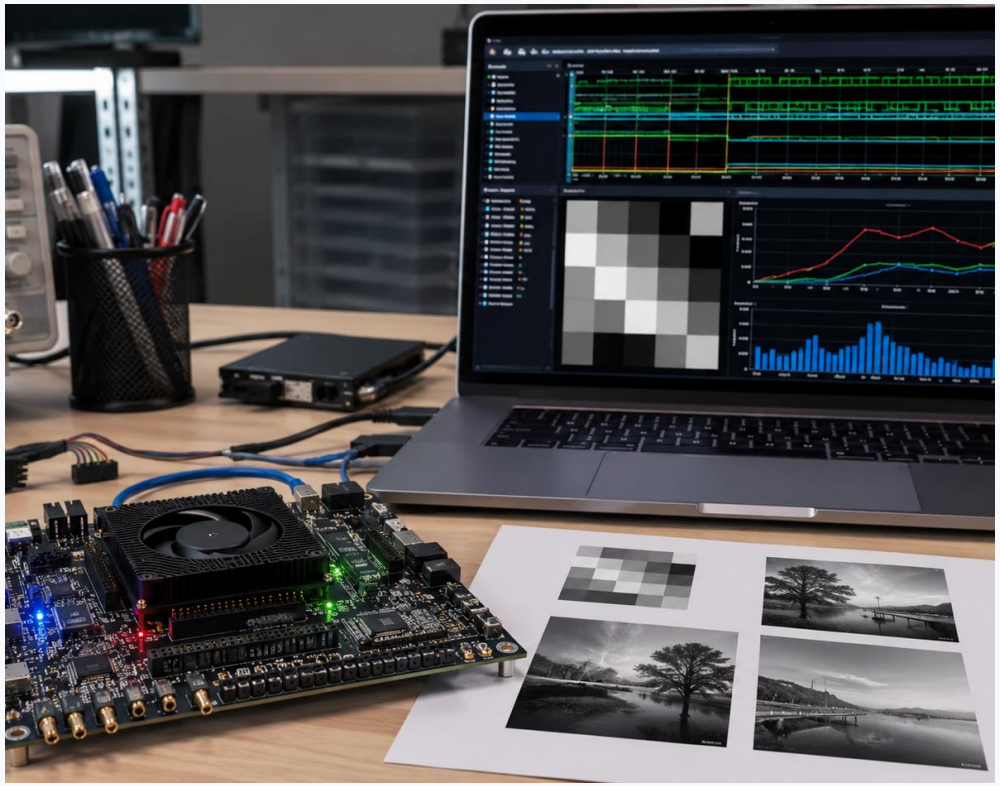
Use: Grayscale PGM/raw images first.

Avoid: RGB, YCbCr, chroma subsampling, Huffman coding, and JPEG headers.

Project-specific aims

A realistic target is a complete compression-side accelerator plus software verification

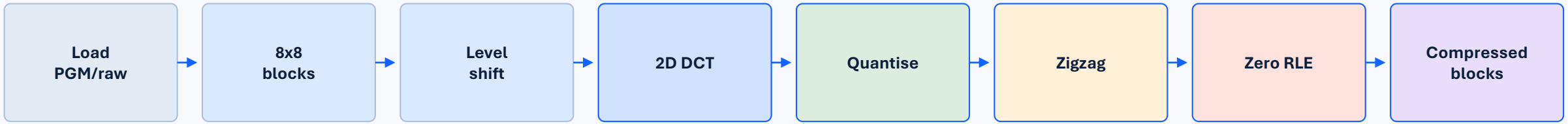
Functional	CPU baseline and FPGA kernel produce compatible quantised/RLE output.
Performance	Measure kernel-only and end-to-end runtimes on the KV260.
Compression	Use RLE pair counts to calculate actual compressed size.
Quality	Use inverse path in software to calculate MSE and PSNR.
Engineering	Show HLS resources: LUT, FF, BRAM, DSP, latency and II.



Primary target: 10x-30x kernel speedup over a naive CPU baseline, with 3x-10x end-to-end if transfer overhead is controlled.

Algorithm to be accelerated

Compression-side pipeline with stage function and hardware opportunity



FPGA accelerated compression kernel

Image blocking

CPU loads fixed grayscale PGM/raw data; FPGA reads 8x8 tiles into local buffers. Acceleration opportunity: regular burst reads and block-level pipelining.

Level shift + DCT

Subtract 128, then perform separable DCT: 1D DCT across rows followed by columns. Main acceleration point: pipelined MACs, loop unrolling, precomputed constants, array partitioning.

Quantise + reorder

Divide each DCT coefficient by an 8x8 quantisation matrix; zigzag uses a fixed lookup order to place low frequencies first and high frequencies last. Acceleration: 64 independent coefficient operations.

RLE + verification

RLE stores zero-run counts and non-zero AC values using fixed 63-pair slots per block. CPU decodes and reconstructs image to calculate PSNR/MSE.

Stage function and acceleration points

How each pipeline stage works, and whether it should run in FPGA

Stage	How it works	Where acceleration is possible
8x8 blocks	Split fixed grayscale image into independent 64-pixel tiles. Each block is processed the same way.	Parallel/block pipeline friendly; local buffers reduce repeated DDR access.
Level shift	Convert unsigned pixels to signed values: pixel - 128, centering the block around zero.	Simple per-pixel operation; pipeline with block loading at II close to 1.
DCT	Convert spatial pixels into 64 frequency coefficients; DC = average brightness, AC = detail.	Main hotspot: use separable row/column DCT, fixed point, unrolled MACs, partitioned arrays.
Quantisation	Divide coefficients by Q matrix; high-frequency values use larger Q and become zeros.	64 coefficient operations can be pipelined/partially unrolled; use constants/reciprocals.
Zigzag scan	Read coefficients in low-to-high frequency order so zeros collect near the end of the list.	Fixed lookup table; low compute cost but can stay inside FPGA to avoid extra CPU pass.
Zero RLE	Store DC plus pairs (zero_run, value) for non-zero AC coefficients; num_pairs marks valid output.	Variable output managed by fixed 63-pair slots; accelerate counting/writing while compression size uses num_pairs.

Priority order for hardware: DCT first, quantisation second, zigzag/RLE third. This keeps a working fallback if integration time is limited.

Acceleration model and expected performance

The best speedup comes from accelerating repeated arithmetic, not file-format handling

Slowest baseline	Naive C++ direct 2D DCT + quantisation + zigzag + RLE on CPU. Useful as the slowest valid reference.
Fair baseline	Optimised CPU separable DCT with precomputed constants. This prevents inflated speedup claims.
FPGA target	Fixed-point separable DCT, pipelined loops, unrolled 8-point MACs, partitioned local arrays, FPGA zigzag/RLE if stable.

Acceleration potential by stage

DCT	**Very high**	Core compute bottleneck; parallel MAC pipelines
Quantise	**Medium**	64 independent coefficient divisions/scales
Zigzag	**Low**	Fixed index reorder; useful to keep on FPGA
RLE	**Medium / risk**	Zero counting is simple; variable output needs fixed slots
Transfers	**Risk**	Measure kernel-only and end-to-end separately

Expected speedup range	**20x-100x vs naive**	**5x-30x vs fair CPU**	**2x-15x end-to-end**	**Target: 3x-10x overall**

Investigation so far and project plan

Four-week implementation path from CPU reference to KV260 measurements

Investigation completed / decisions made

- Use grayscale images to avoid RGB and colour-space complexity.
- Use simplified JPEG-style pipeline rather than full JPEG bitstream.
- Profile each stage separately so we can show where acceleration helps most.
- Use fixed maximum RLE output per block to avoid dynamic memory in HLS.

Planned evidence

- CPU vs FPGA runtime table for DCT, quantisation, zigzag/RLE and full kernel.
- Compression ratio vs PSNR for multiple quantisation scales.
- Resource usage, data-transfer overhead and stage-level acceleration discussion.

Risks, contingencies and team roles

Keep the scope tight: DCT + quantisation first, then zigzag, then RLE

RLE variable output	Use fixed 63-pair slots per block and store num_pairs; report actual compressed size separately from allocated buffer.
DCT numerical mismatch	Start with CPU float reference, then fixed-point HLS. Use tolerances and PSNR rather than exact coefficient equality.
Transfer overhead hides speedup	Measure kernel-only and end-to-end times; test larger images such as 512x512 or 1024x1024 if stable.
Schedule risk	Minimum fallback: FPGA accelerates DCT + quantisation; zigzag/RLE can run in software if integration time is tight.

Proposed team roles

Member 1 **zid**	Software baseline, image I/O, inverse path, PSNR/MSE
Member 2 **zid**	HLS DCT + quantisation kernel and fixed-point design
Member 3 **zid**	Zigzag/RLE format, compression ratio, test vectors
Member 4 **Zid**	KV260 integration, host code, timing, resource/energy results

Final deliverable: a working FPGA-assisted compressor with measured speedup, quality loss, compression ratio, resources, and a clear explanation of limitations.